

Webserv — Person 2 Learning Topics

HTTP + configuration (parser, Request/Response, status codes, headers, error pages)

Role: Milena — HTTP and configuration layer

Owns: configuration parser, HTTP parser, Request object, Response object, status codes, headers, error pages, response formatting

Produces: `HTTPRequest` , `HTTPResponse` , structured config for teammates

Does not: touch sockets, `poll` , routing, GET/POST/DELETE file logic, CGI execution

Person 2 turns **raw bytes** (from Person 1) into a structured **HTTPRequest**, turns a structured **HTTPResponse** (from Person 3, or your own error responses) into a **valid HTTP message**, and loads **server/route rules** from a config file that Person 1 and Person 3 will query.

1. Mental model of your layer

The server is three stacked jobs:

1. **Person 1** moves bytes on sockets.
2. **You** give those bytes meaning (HTTP) and load the rules (config).
3. **Person 3** decides what the server should *do* (files, uploads, CGI, listing).

Your world is **text and structure**, not file descriptors:

- A config file on disk → nested objects (`server` , `location` , listen ports, limits, error pages).
- A raw request buffer → `HTTPRequest` (or a parse error such as 400 / 413 / 505).
- An `HTTPResponse` → one string (or header block + body) that Person 1 can `send` .

Learn the words: **request-line**, **status-line**, **header**, **body**, **CRLF** (`\r\n`), **message framing**, **Content-Length**, **chunked encoding**, **status code**, **reason phrase**, **directive**, **location / route**.

TCP has no message boundaries. Person 1 accumulates bytes. **You** must be able to answer: “is this buffer a complete HTTP request yet? if yes, parse it; if it is garbage, 400; if the body is too big, 413.”

2. Agree the team contract first

Lock these before writing much. They save weeks later.

Boundary	What you need
Person 1 → you	When is a request complete? Who accumulates bytes? Who asks “headers done / body done”?
You → Person 3	Fields of <code>HttpRequest</code> : method, path, query, version, headers, body, content-length, chunked?, max-body exceeded?
Person 3 → you	Fields of <code>HttpResponse</code> : status, headers, body (buffer vs file path — pick one model).
Config → Person 1	Listen address:port list (multiple sockets in one poll).
Config → Person 3	<code>root</code> , <code>index</code> , <code>autoindex</code> , allowed methods, redirect, upload store, <code>client_max_body_size</code> , <code>error_page</code> , CGI extension → binary.
Errors	You: malformed HTTP and bad config. Person 3: routing/file/CGI. You format the final bytes either way.

Also agree: one config format (nginx-like is fine), and that virtual hosts are optional.

3. HTTP message shape (core of your work)

HTTP is an application protocol: a request message and a response message, each made of a start line, headers, a blank line, then an optional body.

What a request looks like

- **Request-line:** `METHOD SP request-target SP HTTP-version CRLF`
Example: `GET /index.html HTTP/1.1`
- **Headers:** `Name: value CRLF` , repeated. Names are **case-insensitive**.
- **Empty line:** `CRLF` (so the header block ends with `\r\n\r\n`).
- **Body:** optional raw bytes. Present for many POSTs; usually absent for GET/DELETE.

What a response looks like

- **Status-line:** `HTTP-version SP status-code SP reason-phrase CRLF`
Example: `HTTP/1.1 404 Not Found`
- Headers + blank line + optional body (error HTML, file contents, CGI output, etc.).

What to study in the RFCs

You are not implementing the entire RFC. Read enough of:

- **RFC 2616** (HTTP/1.1, dense, classic for 42), or the clearer split: **RFC 7230** (message syntax / framing) and **RFC 7231** (semantics: methods, status codes).
- HTTP/1.0 (RFC 1945) as a simpler reference point — the subject suggests it but does not enforce it.

Focus only on:

- request-line / status-line

- headers (case-insensitive names; obsolete line-folding — many 42 servers skip folding)
- `Content-Length`
- **chunked transfer encoding** (you must **dechunk** before Person 3 / CGI sees the body)
- absolute path + query string (`/path?a=1`)
- connection basics (`Connection: close` is enough for a solid mandatory part)

Practice against nginx (mandatory habit)

Use `telnet` , `curl -v` , and a browser. Compare **status codes and headers**, not pixel-perfect clones:

- GET with and without query string
- POST with body + `Content-Length`
- POST with `Transfer-Encoding: chunked`
- oversized body
- unknown method
- bad request-line / missing blank line / binary junk

4. Request-line details

Three tokens separated by spaces. Anything else is 400.

Method

Subject requires at least **GET, POST, DELETE**. You still need to *parse* whatever token is there and store it. Unknown methods are often **501** (not implemented) or **405** (not allowed on this route — that decision is Person 3 using your config). Agree the split: you reject syntactically invalid methods (empty, spaces, control chars) with 400; Person 3 rejects disallowed methods with 405.

Request-target / URI

Typical form: origin-form `/path` or `/path?query` .

- Split **path** and **query** at the first `?` .
- You may percent-decode (`%20` → space). Learn when: path decoding vs leaving query raw for CGI `QUERY_STRING` .
- Path normalization (`/./` , `//` , `/../`) is security-sensitive. Often Person 3 owns “must not escape root ”, but you may offer a normalize helper. Agree this explicitly.
- Absolute-form (`http://host/path`) and asterisk-form (`*`) exist in the RFC; origin-form is enough for browsers. Decide whether extra forms are 400.

Version

Expect `HTTP/1.0` or `HTTP/1.1` . Other versions → **505** (HTTP Version Not Supported). Do not crash on `HTTP/2.0` or garbage.

5. Headers

This is where browsers look “messy but valid”.

- Format: `field-name ":" OWS field-value OWS` . OWS = optional spaces/tabs.
- Names are case-insensitive: `Content-Length` , `content-length` , `CONTENT-LENGTH` are the same header. Store them in a way Person 3 can look up without caring about case.
- Duplicate headers: some combine with commas; `Host` duplicated is often 400. Learn a simple policy and stick to it.
- You need reliable getters: `Content-Length` , `Content-Type` , `Transfer-Encoding` , `Host` , `Connection` , `Cookie` (bonus).
- `Host` is required in HTTP/1.1. Missing `Host` → 400 is RFC-correct and fine to implement even without virtual hosts.
- Do not assume a maximum header count or size unless you define one (protection against slowloris / memory blow-up). A max header block size that yields 400 or 431 is wise.

Person 3 and CGI will need **all** headers, not a whitelist. CGI maps them to `HTTP_*` environment variables.

6. Body rules (the hard part of your lane)

After `\r\n\r\n` , you must know how many more bytes belong to this request. Two legal ways in this project:

Content-Length

- Value must be a non-negative integer. Junk → 400.
- Read exactly N bytes as the body.
- If the buffer has fewer than N bytes, the request is **incomplete** (Person 1 keeps reading).
- If N is larger than `client_max_body_size` → **413** (Payload Too Large). You can reject as soon as the header is known; you do not have to buffer the whole illegal body.

Chunked transfer encoding

The subject is explicit: for chunked requests, the server **un-chunks** them. CGI expects a raw body and EOF, not chunk syntax.

Learn the format:

- Each chunk: hex size, `CRLF` , that many bytes, `CRLF` .
- Last chunk: size `0` , then optional trailers, then `CRLF` .
- After decoding, set the real body length. CGI `CONTENT_LENGTH` must match the **decoded** body.

Malformed chunks → 400. Running decoded size over max body → 413.

No length and no chunked

For GET/DELETE, body is usually empty (you may ignore leftover bytes or 400 — pick a policy). For POST without length info, HTTP/1.1 cannot know where the body ends → typically 400. Do not hang waiting forever.

Completion checker (Person 1 depends on this)

Expose a clear answer for a growing buffer:

- **Need more bytes** (headers not finished, or body incomplete).

- **Complete** → parse into `HTTPRequest` .
- **Error** → status 400 / 413 / 505, and Person 1 should stop reading that request.

This completion logic is where many groups fail evaluation. Design it with Person 1. You still never call `recv` .

7. Status codes you must get right

The subject: “Your HTTP response status codes must be accurate.” Learn meaning and typical trigger:

Code	Name	Typical owner / trigger
200	OK	Person 3 — successful GET
201	Created	Person 3 — successful upload / POST that created a resource
204	No Content	Person 3 — success with empty body (some DELETes)
301 / 302 / 307 / 308	Redirects	Person 3 using your <code>return</code> / redirect config. Pick which codes you support.
400	Bad Request	You — broken request-line, headers, chunks
403	Forbidden	Person 3 — no permission / listing disabled
404	Not Found	Person 3 — missing file
405	Method Not Allowed	Person 3 — method not in location’s allow list. Response should include <code>Allow</code> .
408	Request Timeout	Person 1 detects stall; you format the response
413	Payload Too Large	You — body exceeds <code>client_max_body_size</code>
500	Internal Server Error	Person 3 — CGI crash, unexpected failure. You may also use it if serialization fails.
501	Not Implemented	Unknown method / unsupported feature
505	HTTP Version Not Supported	You — not 1.0/1.1

Learn the **reason phrase** that goes on the status line (`Not Found` , `Bad Request` , ...). Browsers mostly use the number; evaluation and curl show the text.

8. HTTPResponse and serialization

Person 3 fills a response object. You turn it into bytes.

A valid response needs:

1. Status line with correct version, code, reason phrase.

2. Headers Person 3 set, plus ones you should guarantee:

- `Content-Length` (almost always, unless you deliberately close the connection to end the body)
- `Content-Type` (HTML error pages, static files — Person 3 may set MIME; you can provide a helper)
- `Connection: close` (simple and safe for mandatory)
- `Location` on redirects
- `Allow` on 405
- `Date` is optional but looks professional

3. Blank line.

4. Body (omit for 204; error pages for 4xx/5xx if Person 3 did not supply a body).

Decide one body model with Person 3:

- **Always a string in memory** — simplest. Person 3 reads the file, you format.
- **Body is a file path** — Person 1/3 stream the file. You only format headers. Harder, better for large files.

Start with in-memory body. Large-file streaming can come later if needed.

9. Default error pages

Subject: the server must have **default error pages** if none are provided in config.

Learn / decide:

- A small built-in HTML page per status (400, 403, 404, 405, 413, 500, ...).
- Config `error_page 404 /404.html`; means: if that file exists under the right root, use it; otherwise fall back to the built-in page.
- Who reads the custom file? Disk reads are allowed without poll. Either you load it when building the error response, or Person 3 does. Agree one owner so pages are not missing during eval.
- Error pages still need a correct status line. Serving a 404 HTML with status 200 is a classic fail.

10. MIME types and small HTTP helpers

Not “business logic”, but Person 3 will need them:

- Map extension → `Content-Type` (`.html`, `.css`, `.js`, `.png`, `.jpg`, `.ico`, `.txt`, ...). Unknown → `application/octet-stream`.
- Query-string split into key/value pairs (CGI and forms).
- Header get/set with case-insensitive keys.
- Optional: cookie parsing (bonus only).

Keep routing, autoindex HTML generation, upload saving, and CGI `fork` **out** of your ownership.

11. Configuration file (start here in the build order)

Subject: the program takes `./webserv [configuration file]`, or a default path. You cannot exec another web server. Nginx `server` / `location` is the model, not a clone.

What the config must be able to express

- All **interface:port** pairs to listen on (multiple websites / ports).
- Default **error pages**.
- **Maximum client body size**.
- Per-URL / route (no regex required) rules:
 - accepted HTTP methods
 - HTTP redirection
 - directory (root alias) where files live — e.g. URL `/kapouet` rooted at `/tmp/www` means `/kapouet/pouic/toto/pouet` → `/tmp/www/pouic/toto/pouet`
 - directory listing on/off (autoindex)
 - default file for a directory (index)
 - upload allowed + storage location
 - CGI by file extension (e.g. `.php` → interpreter path)

You may add extras (`server_name` if you do virtual hosts).

What to study

- Nginx docs for `listen`, `server`, `location`, `root`, `alias`, `index`, `error_page`, `client_max_body_size`, `limit_except` / allowed methods, `return`, `autoindex`, CGI/fastcgi as inspiration only.
- A **grammar**: braces, semicolons, comments, nested blocks, quoted vs unquoted values, size units (`10m`, `100k`).
- A **data model** Person 3 can walk: list of servers → list of locations. Usual match: **longest prefix** of the request path (no regex).

Validation (fail at startup, not at the first request)

- Missing file, empty file, unmatched braces, unknown directive (or ignore unknown — pick a policy).
- No listen port, duplicate listen in a confusing way, invalid port, bad size, empty method list, missing root where needed, CGI extension without binary.
- Invalid config → clear error message on stderr and exit. Do not “half run”.

Person 1 needs listen addresses as soon as the process starts. Person 3 needs locations before any request. Your parser unblocks both.

12. CGI contract (you prepare the request; Person 3 runs it)

Read CGI/1.1 environment variables conceptually. You do not `fork`. You must make sure `HTTPRequest` is complete enough:

- `REQUEST_METHOD`, `SCRIPT_NAME`, `PATH_INFO`, `QUERY_STRING`
- `CONTENT_LENGTH`, `CONTENT_TYPE`
- `SERVER_NAME`, `SERVER_PORT`, `SERVER_PROTOCOL`, `REMOTE_ADDR` (some of these Person 1 must supply: port, client IP)
- `HTTP_*` for each header

Subject remarks you must understand:

- Full request and client arguments available to the CGI.
- Chunked requests are un-chunked first; CGI sees EOF as end of body.
- CGI output: if no `Content-Length` from CGI, EOF ends the data. Person 3 reads the CGI; you may help parse CGI headers (`Status:` , `Content-Type:`) vs HTTP headers. Agree who translates CGI output into `HTTPResponse` .

13. C++98 tools for your layer

No Boost, no C++11. Refresh:

- `std::string` , `std::vector` , `std::map` / `std::list`
- `std::stringstream` for scanning tokens if you like
- `<cctype>` for character checks; prefer `<cstring>` over `<string.h>`
- No need for a huge framework. Small classes: `Config` , `ServerConfig` , `Location` , `HTTPRequest` , `HTTPResponse` , `HttpParser` , `ResponseBuilder`
- Do not crash on bad input: return error responses or fail config load cleanly (subject: never crash, even out of memory)
- Makefile: `c++ -Wall -Wextra -Werror -std=c++98`

You never need `socket` , `poll` , `recv` , `send` . Disk `open` / `read` for config and error pages is allowed without `poll`.

14. Recommended build order (how to start writing)

Each phase should be testable with fake strings, without the real server.

Phase 0 — Design notes only

Write down: config schema, request fields, response fields, error ownership. Sync with Person 1 and Person 3 once.

Phase 1 — Config parser (start here)

1. Define the grammar you accept (nginx subset).
2. Parse into nested structs.
3. Validate (ports, sizes, methods, paths).
4. Fail at startup with a clear message if invalid.

Deliverable: load a `.conf` → dump structured config for debugging. Person 1 can then bind ports.

Phase 2 — HTTPRequest + request parser

Input: one complete raw string.

1. Split headers / body at `\r\n\r\n` .
2. Parse request-line → method, target, version.
3. Parse headers into a case-insensitive map.

4. Decide body length (Content-Length vs chunked).
5. Reject garbage with 400.

Deliverable: sample request files → filled `HttpRequest` or parse error.

Phase 3 — Body rules

1. Content-Length extraction.
2. Chunked decoding → raw body.
3. Enforce `client_max_body_size` → 413.
4. Incomplete vs complete (so Person 1 knows when to stop reading).

Phase 4 — HTTPResponse + serializer

1. Status line.
2. Headers including Content-Type, Content-Length, Connection, Location.
3. Body.
4. Default error HTML.
5. Optional custom error page from config.

Deliverable: `HTTPResponse` → bytes Person 1 can send.

Phase 5 — Glue helpers for Person 3

Header helpers, MIME map, query split, optional path normalize. No routing.

Phase 6 — Integration

Person 1 accumulates → your “is complete?” → parse → Person 3 → your serialize → send. Test with browser, curl, telnet, stress. Fix status codes vs nginx.

Phase 7 — README (whole team, subject-required)

English README, italic first line with 42 logins, Description, Instructions, Resources, and how AI was used. Near the end.

15. How to practice without waiting on teammates

1. Collect a folder of **raw request dumps** (curl verbose / telnet captures / browser).
2. Tiny tests: parse string → assert method, path, headers, body.
3. Craft bad requests: no Host, bad chunks, huge Content-Length, missing CRLF, HTTP/2, binary.
4. Write a small nginx conf similar to yours; compare status and important headers.
5. Write example configs that demonstrate every subject feature (eval requires this).
6. Dump parsed config and parsed requests to stdout until you trust them.

16. Subject landmines specific to Person 2

These will show up in evaluation even if Person 1's sockets are perfect.

- Parsing before the request is complete → false 400s or hangs (work with Person 1).
- Header names treated as case-sensitive.
- Leaving the body still chunked when Person 3 / CGI expects raw bytes + correct length.
- Wrong status codes (404 page with 200, 405 without understanding allow list vs 400).
- No default error pages.
- Config parser that accepts anything and crashes later in Person 3's code.
- Missing `client_max_body_size` enforcement (413).
- Server hangs because you never declare “incomplete” vs “done”.
- Crashing on malformed input or huge headers (must not crash).
- Building a giant framework instead of parse → object → format.

17. Suggested learning order (topics checklist)

1. HTTP message anatomy: start line, headers, blank line, body, CRLF
2. Request-line: method, path, query, version
3. Headers: case, lookup, Host, Content-Length, Transfer-Encoding, Connection
4. When a request is complete (framing) — your contract with Person 1
5. Content-Length bodies
6. Chunked encoding and dechunking
7. Status codes and reason phrases
8. Response serialization and Content-Length on the way out
9. Default + custom error pages
10. Nginx-like config grammar and nested server/location model
11. Config validation and listen ports for Person 1
12. Location fields Person 3 will query (root, index, methods, redirect, upload, CGI, autoindex, max body)
13. CGI env expectations so your Request is complete
14. MIME types and small helpers
15. Nginx comparison testing with curl/telnet/browser
16. C++98 string/map parsing without forbidden libraries

18. What Person 2 does not need to master first

Skip or postpone:

- `poll`, non-blocking sockets, `accept` (Person 1)
- Routing algorithm, autoindex HTML, saving uploads, `fork / execve` CGI (Person 3)
- HTTP/2, HTTPS/TLS, WebSockets
- Full RFC every appendix; cookies/sessions until bonus
- Threads, Boost, libcurl as a server

A strong Person 2 is someone who can explain, without code: **where an HTTP message starts and ends, why header names are case-insensitive, why chunked bodies must be decoded before CGI, and how a nginx-like config becomes a list of listen ports plus per-route rules.**

19. Suggested first week

1. **Day 1–2:** RFCs (framing + semantics skim) + telnet/nginx experiments; write down request and response field lists.
2. **Day 2–3:** agree interfaces with Person 1 and Person 3.
3. **Day 3–5:** config grammar + parser + validation.
4. **Day 5–7:** request-line + headers parser (bodies the following week).